



NATIONAL UNIVERSITY OF SCIENCES AND
TECHNOLOGY

SCHOOL OF MECHANICAL AND MANUFACTURING
ENGINEERING

RAI-833 Deep Learning

Tutorial 13: Advanced Autoencoders

Feature Restoration (DAE) & Generative Modeling (VAE)

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

May 16, 2026

1 Objectives

Expanding upon unregularized representation learning frameworks, this tutorial introduces structural input corruption and probabilistic latent spaces to prevent identity memorization during unsupervised training. Key practical and theoretical objectives include:

- Formulating programmatic pipelines to dynamically inject additive **Gaussian Noise** directly into baseline input tensors using **PyTorch**.
- Implementing symmetric multi-layer **Denoising Autoencoders (DAEs)** optimized to reconstruct target ground-truth representations directly from degraded feature vectors.
- Implementing **Variational Autoencoders (VAEs)** to learn continuous, structured latent spaces via the **Reparameterization Trick**.
- Evaluating the generative capabilities of VAEs by autonomously synthesizing novel, high-fidelity image morphologies from pure random standard normal noise.

2 Methodology

Standard autoencoders risk learning raw identity matrices if internal bottleneck layer capacities exceed intrinsic data dimensions. We explore two distinct methodologies to enforce robust manifold learning.

2.1 Denoising Autoencoders (DAEs)

The DAE workflow requires an initial stochastic corruption mapping followed by standard parametric encoder-decoder projections. An uncorrupted source sample $x \in \mathbb{R}^d$ is mapped to a corrupted variant $\tilde{x}$ utilizing an additive Gaussian profile:

$$\tilde{x} = \text{clamp}(x + \sigma \cdot \mathcal{N}(0, \mathbf{I}), 0, 1) \quad (1)$$

To enforce feature extraction over pixel memorization, network weights are updated computing loss metrics exclusively comparing predicted restorations against original uncorrupted source targets (x), utilizing **Mean Squared Error (MSE)**.

2.2 Variational Autoencoders (VAEs)

Unlike standard deterministic autoencoders, VAEs map inputs to a probabilistic distribution, enabling true generative AI capabilities.

- **Reparameterization Trick:** To allow backpropagation through stochastic sampling, the latent vector z is sampled using learned mean (μ) and log-variance ($\log(\sigma^2)$) parameters:

$$z = \mu + \epsilon \odot \sigma \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}) \quad (2)$$

- **Optimization (ELBO):** The network minimizes the Evidence Lower Bound, balancing pixel-wise reconstruction accuracy (Binary Crossentropy) with **Kullback-Leibler (KL) Divergence** to force the latent space into a continuous standard normal distribution:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{BCE}} - \frac{1}{2} \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2) \quad (3)$$

3 Implementation Code

Below is the PyTorch implementation detailing the custom VAE module utilizing the reparameterization trick and the hybrid ELBO loss framework.

```
1 import torch
2 import torch.nn.functional as F
3 from torch import nn
4
5 class VAE(nn.Module):
6     def __init__(self):
7         super(VAE, self).__init__()
8         self.fc1 = nn.Linear(28*28, 400) # Encoder Base
9         self.fc21 = nn.Linear(400, 20)   # Latent Mean (mu)
10        self.fc22 = nn.Linear(400, 20)   # Latent Log Variance (logvar)
11        self.fc3 = nn.Linear(20, 400)    # Decoder start
12        self.fc4 = nn.Linear(400, 28*28) # Decoder output
13
14        def encode(self, x):
15            h1 = F.relu(self.fc1(x))
16            return self.fc21(h1), self.fc22(h1)
17
18        def reparameterize(self, mu, logvar):
19            std = torch.exp(0.5 * logvar)
20            eps = torch.randn_like(std)
21            return mu + eps * std
22
23        def forward(self, x):
24            mu, logvar = self.encode(x.view(-1, 28*28))
25            z = self.reparameterize(mu, logvar)
26            decoded = torch.sigmoid(self.fc4(F.relu(self.fc3(z))))
27            return decoded.view(-1, 1, 28, 28), mu, logvar
28
29        def vae_loss_function(recon_x, x, mu, logvar):
30            BCE = F.binary_cross_entropy(recon_x.view(-1, 28*28), x.view(-1,
31            28*28), reduction='sum')
32            KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
33            return BCE + KLD
```

Listing 1: PyTorch implementation of the Variational Autoencoder and custom KL-Divergence loss.

4 Results and Discussion

The architectures were optimized end-to-end on image distributions using an initial Adam optimization profile ($lr = 10^{-3}$).

4.1 Comprehensive Model Comparison

To evaluate the baseline reconstruction capabilities of both architectures concurrently, the models were tested against a standard batch of MNIST digits. As illustrated in Figure 1, the Deep Denoising Autoencoder (Row 3) successfully isolates and strips heavy Gaussian noise artifacts (Row 2) to restore the original clean digit structures (Row 1). In parallel, the Variational Autoencoder (Row 4) reconstructs the clean inputs by passing them

through its probabilistic latent space, demonstrating the slight morphological smoothing that is mathematically characteristic of continuous VAE sampling.

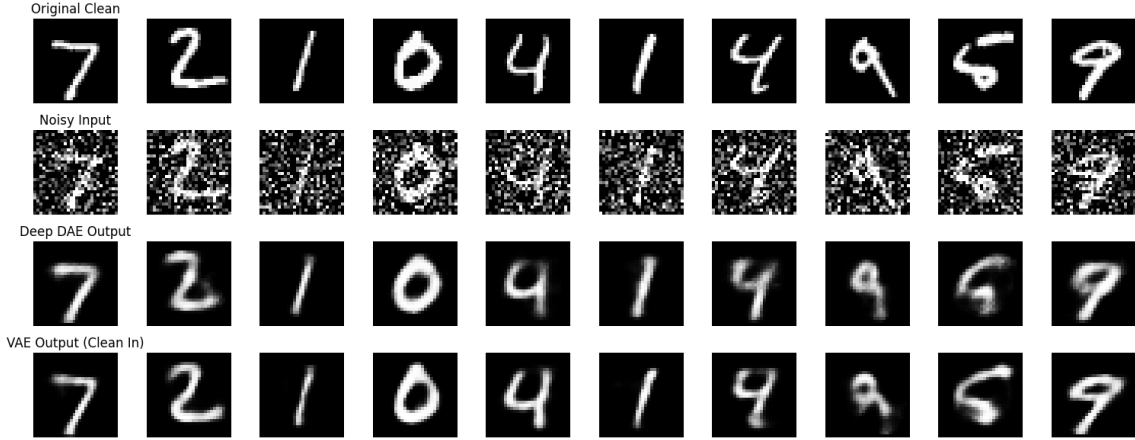


Figure 1: Side-by-side architectural reconstruction comparison. Row 1: Original Clean Inputs. Row 2: Corrupted Inputs (Gaussian Noise). Row 3: Deep DAE Restoration Output. Row 4: VAE Reconstruction Output.

4.2 Task 1: Denoising Restoration Resiliency

Evaluating the network trained under standard corruption profiles demonstrated stable manifold convergence. As mapped in Figure 2, the DAE effectively subtracts continuous granular noise artifacts while successfully articulating clean baseline topological silhouettes across diverse object classes.

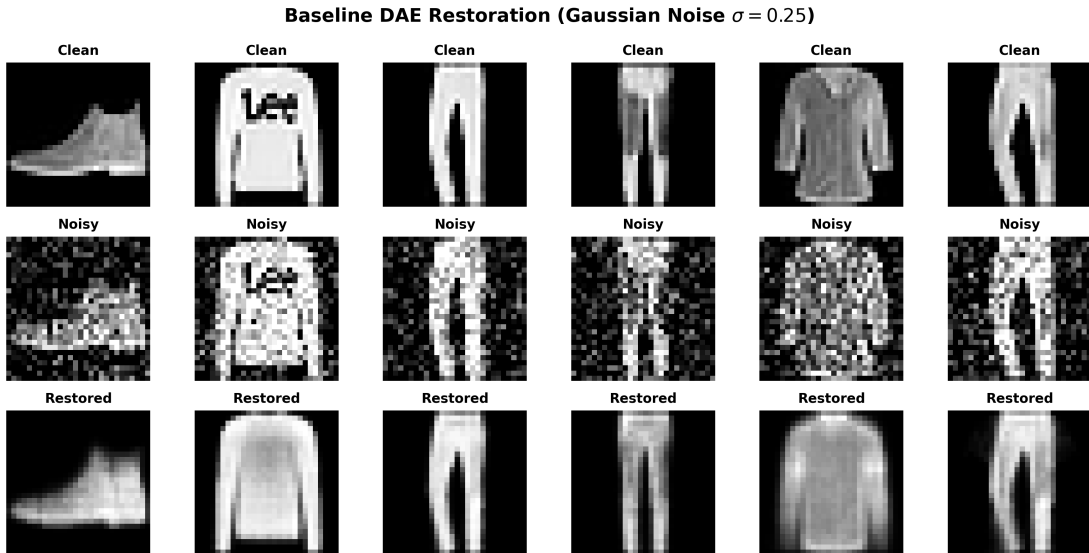


Figure 2: Baseline model inference evaluation. Corrupted visual coordinate representations (Middle Row) are cleanly projected back to their underlying spatial manifolds to generate high-fidelity output silhouettes (Bottom Row).

4.3 Task 2: VAE Latent Space Organization

To fulfill the secondary tutorial requirements, a Variational Autoencoder was implemented to explore organized latent topologies. By enforcing a Gaussian prior via KL Divergence, the VAE naturally clusters similar digits into continuous mathematical regions without utilizing any explicit class labels during training.

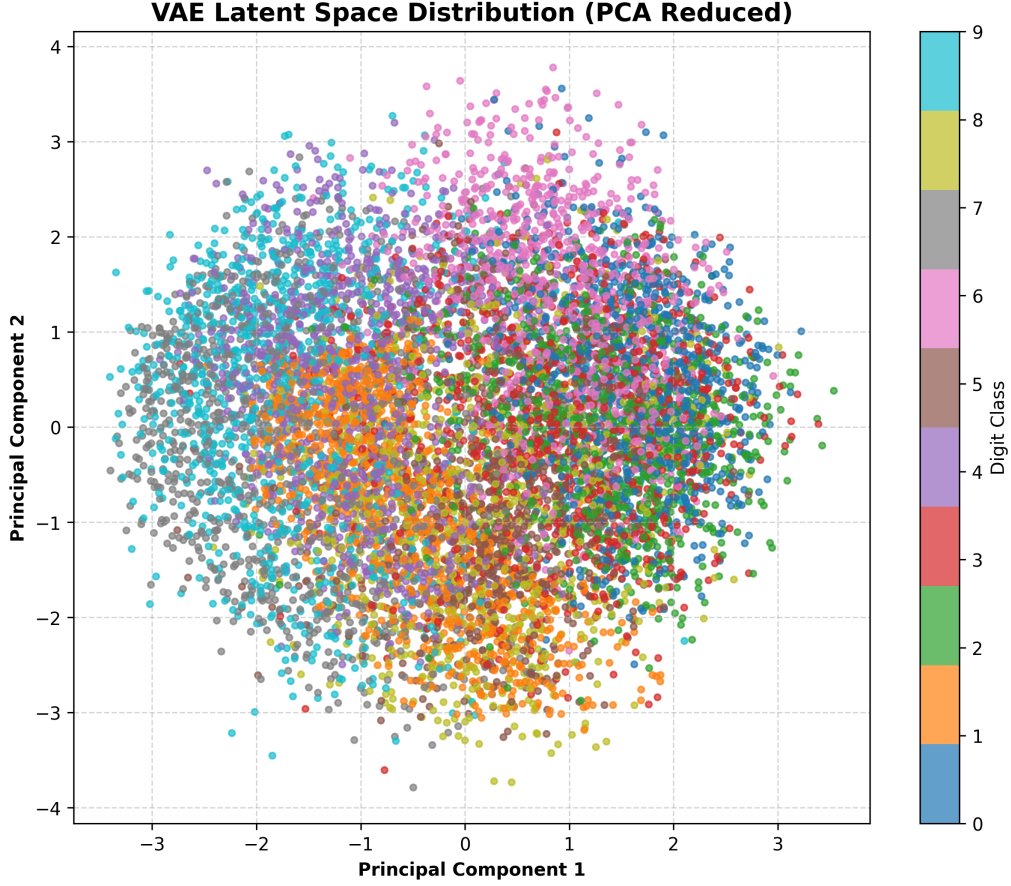


Figure 3: Principal Component Analysis (PCA) projection of the 20-dimensional VAE latent space. The network autonomously clusters morphological digit classes (e.g., separating round '0's from linear '1's) into distinct, continuous manifolds.

4.4 Generative Synthesis via Random Sampling

Because the VAE latent space is continuous and normally distributed, the decoder can act as an independent Generative AI model. By bypassing the encoder entirely and feeding random mathematical noise $\mathcal{N}(0, \mathbf{I})$ directly into the decoder, the network successfully synthesizes entirely novel outputs.

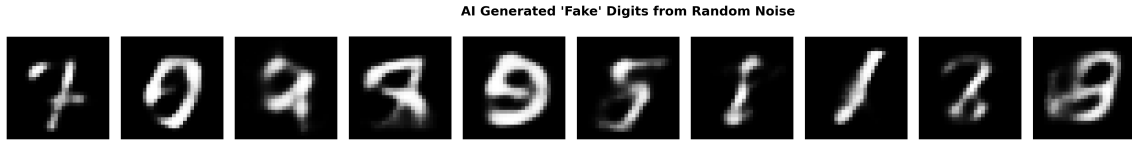


Figure 4: AI-generated digits synthesized entirely from random standard normal noise distributions. These specific handwriting morphologies do not exist in the training dataset, demonstrating the VAE’s true generative capabilities.

5 Conclusion

This tutorial successfully demonstrated the power of advanced self-supervised representation learners. The DAE empirical outputs prove that networks trained against uncorrupted targets learn robust internal feature descriptors capable of projecting highly degraded images back onto standard uncorrupted manifolds. Furthermore, the implementation of the Variational Autoencoder illustrated the critical leap from feature restoration to generative synthesis. By utilizing the reparameterization trick and KL-Divergence penalties, the VAE forged a continuous, explorable latent space, allowing the decoder to independently generate complex, realistic data purely from randomized vector noise.

